

B.Sc. (ECS)-II

Software Testing and Quality Assurance

Unit-1

Introduction To Software Testing

Software testing is an important process in the Software Development Lifecycle (SDLC). Software Testing is a process of verifying and validating whether the Software Product or Application is working as expected or not.

Software testing is carried out in a systematic manner to find defects in a system.

Software Testing Can be Divided into Two Steps

- 1) Verification
- 2) Validation

1) Verification:

This step involves checking if the software is doing what is supposed to do. Its like asking, "Are we building the product the right way?"

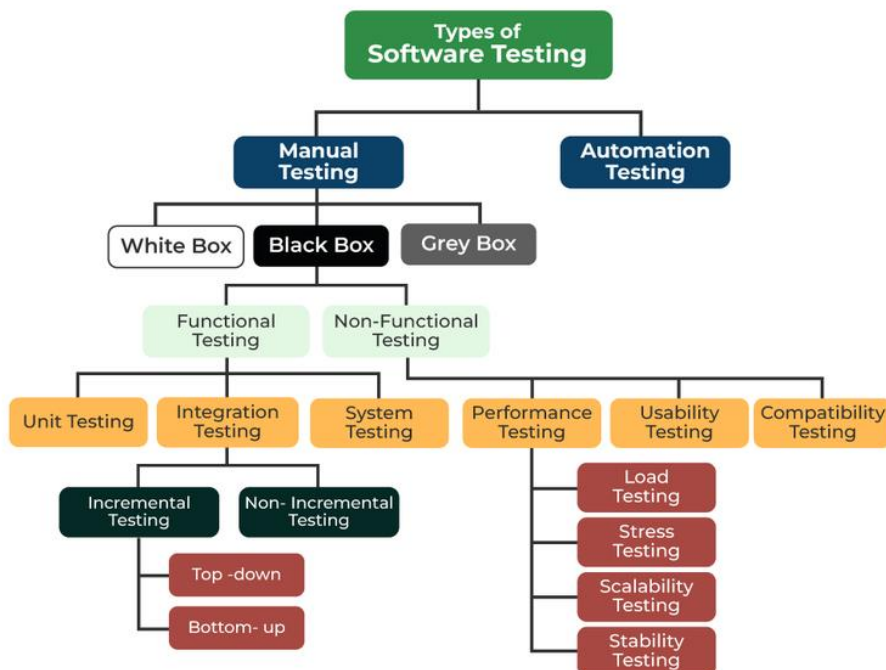
2) Validation:

This step verifies that the software actually meets the customer's needs and requirements. Its like asking, "Are we building the right product?"

Types of Software Testing

Basically, there are two types of software testing as,

- 1) Manual Testing
- 2) Automation testing



1) Manual Testing:

In manual testing we test the software by checking the defect manually with trying one by one function is working as expected or not.

2) Automation testing:

In Automation testing the tester writes scripts and uses suitable Software or Automation Tools to test the software.

These types of testing further divided into other sub types, we will see them later.

Importance or need of software testing

1. Saves Money

Software testing offers numerous advantages, with cost-effectiveness that attracts companies to opt for testing services.

Discovering bugs(errors) early means they can be fixed at a lower cost.

2. Security

There are a group of conditions in which the statistics and information of the users are stolen and they are used for the benefits.

It is considered to be the motive why humans look for nicely tested and reliable products.

Selected product undergoes testing, and the consumer can be ensured that they are going to receive a dependable product.

The non-public information of the user may be secure.

3. Quality of the product

To make sure that the unique product involves life, it needs to work by the following.

Products bring the price, as according to the promise.

Hence, it ensuring a powerful purchaser enjoys.

It is also important to check the compatibility of the device.

4. Satisfaction of the consumer

The primary goal of the proprietor of the products is to provide the pleasant pride of the customers.

The reasons why it's necessary to opt for software testing is because it offers the perfect consumer satisfaction.

5. Enhancing the development technique

With the aid of Quality Assurance, we can find a big selection of situations and mistakes, for the reproduction of the error.

It is really simple and the builders want to repair the same very quickly. In addition to this, software testers need to be operating with the development team parallelly, which is beneficial in the acceleration of the improvement method.

6. Determining the performance of the software program

If we discover software program or software that has low or reduced overall performance, we will find that it brings our recognition down in the marketplace.

By prioritizing software trying out, businesses can supply dependable, stable, and amazing merchandise that meet person needs and exceed expectancies in brand new competitive market.

Difference between Manual Testing and Automation Testing:

Criteria	Manual Testing	Automation Testing
Accuracy	Manual Testing shows lower accuracy due to the higher possibility of human errors.	Automation Testing shows a higher accuracy due to computer-based testing .
Testing at Scale	Manual Testing needs time when testing is needed at a large scale.	Automation Testing easily performs testing at a large scale.
Turnaround time	Manual Testing takes more time to complete a testing cycle, and thus the turnaround time is higher.	Automation Testing completes a testing cycle within record time; thus, the turnaround time is much lower.
Cost Efficiency	Manual Testing requires more cost as it involves hiring expert professionals.	Automation Testing saves costs incurred as once the software infrastructure is integrated; it works for a long time.
User Experience	Manual Testing ensures a high-end User Experience to the software's end user, as it requires human observation and cognitive abilities.	Automation Testing cannot guarantee a good User Experience since the machine lacks human observation.
Areas of Specialization	To exhibit the best results, manual Testing should be used to perform Exploratory, Usability, and Ad-hoc Testing.	Automation Testing should be used to perform Regression Testing, Load Testing, Performance Testing, and Repeated Execution for best results.
User Skills	Users must be able to copy user behavior and build test plans to cover all the scenarios.	Users must be highly skilled at programming and scripting to build test cases and automate as many scenarios as possible.

Introduction to White Box Testing

We know that, In manual testing we test the software by checking the defect manually with trying one by one function is working as expected or not.

The manual testing can be divided as,

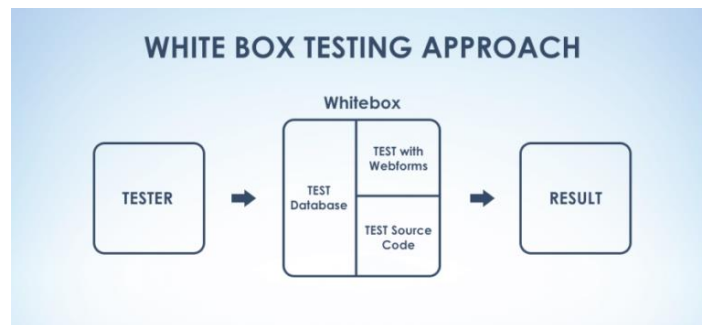
- a) White Box Testing b) Black Box Testing c) Gray Box Testing

Now we are going to see about White box testing

a) White Box Testing

White box testing is a way of testing software by looking inside the code to ensure it works correctly. Testers check how the code is written, how it runs, and whether each part does what it's supposed to. This helps find bugs early and makes the software more reliable and secure.

White box testing is also known as Structural Testing or Code-based Testing, and it is used to test the software's internal logic, flow, and structure.



The tester has access to the source code and uses this knowledge to design test cases that can verify the correctness of the software at the code level.

White-box Testing focuses on the internal workings of an application, ensuring that its logic, structure, and flow operate as per plan.

Advantages of White box testing:

- 1) White box testing is comprehensive and detailed as it exercises every line of the source code.
- 2) It identifies potential hidden errors, defects, and security vulnerabilities. Fixing them requires removing some lines of source code, which results in code optimization.
- 3) It ensures that the source code complies with the coding standards and is performance-optimized. Even if the GUI is unavailable, testing starts early in the software development life cycle (SDLC).
- 4) Test cases are easy to automate.
- 5) The source code transparency helps determine the exact type of input data required for testing.
- 6) Testers or developers can create test cases that can ensure maximum test coverage.

Disadvantages of White box testing:

- 1) white box testing requires in-depth programming knowledge to understand and analyze a system's source code and create test cases around it.
- 2) It primarily focuses on testing the system's internal workings and misses out on external issues.
- 3) Large applications require a lot of time to undergo white box testing due to their lengthy source codes.
- 4) A small change in the source code requires writing test cases again.
- 5) There are strong chances of resulting in production errors.

White box Testing techniques:

There are following two types of White Box Testing(WBT) techniques as,

- 1) Static Testing
- 2) Dynamic Testing

1) Static Testing/Static Techniques:

In this techniques the code is checked manually, it does executed.

Also, in this technique Required documents, design documents and work documents tested manually.

Objectives of Static Testing

Below are some of the objectives of static testing:

Decreases bugs: Static testing will decrease the bugs in production as the bugs will be detected early in the software development.

Saves Time: Early detection of bugs helps to save time, effort, and cost that will be required to fix the bugs.

Easy Bug Fixing: Static testing is used to identify the bugs early in software development, where it is quite easy to fix the bugs.

Quality Improvement: Enhance overall software quality of the software product by ensuring the compliance with the coding standards and best practices.

Cost Efficiency: Static testing helps to reduce the cost associated with dynamic testing by catching defects early.

Features Tested in Static Testing

Static testing involves testing the following things:

Unit Test Cases: It ensures test cases are complete, written in the correct manner, and follow the specified standards.

Business Requirements Document (BRD): It verifies that all business requirements are clearly mentioned in the documentation.

Use Cases: It examines the use cases so that they accurately represent user interactions with the system.

Prototype: It reviews the prototype to make sure that it accurately represents the main design and functionality.

System Requirements: It check the system requirements document for complete accuracy.

Test Data: It reviews the test data to ensure it is complete and covers all possible input scenarios.

Traceability Matrix Document: This ensures that all requirements are mapped to corresponding test cases.

Training Guides: It reviews training materials to make sure that they accurately reflect the system functionality and user procedures.

Performance Test Scripts: It examines performance test scripts to ensure they cover all critical performance aspects.

Static Testing Techniques

There are mainly two types of techniques used in Static Testing:

1. Review

In static testing, the review is a process or technique that is performed to find potential defects in the design of the software. It is a process to detect and remove errors and defects in the different supporting documents like software requirements specifications. People examine the documents and sorted out errors, redundancies, and ambiguities. Review is of four types:

Informal Review: In an informal review the creator of the documents put the contents in front of an audience and everyone gives their opinion and thus defects are identified in the early stage.

2. Walkthrough: It is basically performed by an experienced person or expert to check the defects so that there might not be problems further in the development or testing phase.

3. Technical review: Technical reviews are a static software testing technique focused on examining software artifacts, such as requirements specifications, design documents, and code, without executing the program. It is basically done in a team of colleagues.

4. Inspection: Inspection is basically the verification of documents by the higher authority like the verification of software requirement specifications (SRS).

How Static Testing is Performed?

Below are the steps that can be followed to perform static testing:

Planning: This step involves defining what needs to be tested, setting objectives, determining the scope of testing, and preparing a testing strategy. This should involve identifying the software components to be tested, developing the testing methods, and identifying the tools to be tested.

Prepare artifacts: In this step, necessary artifacts like source codes, design documents, requirement documents, and test cases are prepared.

Perform static analysis: Static analysis is conducted in this phase where the code is reviewed and analyzed for compliance with coding standards, code quality, and security issues using specialized static analysis tools without executing the code.

Perform code reviews: Code reviews are performed where a small team of experts systematically reviews the code and finds potential errors using various methods.

Report and document bugs: Bugs identified during static testing are reported and documented.

Analyze results: The results collected during static testing are analyzed to determine the quality of the software product.

Benefits of Static Testing

Below are some of the benefits of static testing:

Early defect detection: Static testing helps in early defect detection when they are most easy and cost-effective to fix.

Prevention of common issues: Static testing helps to fix common issues like syntax errors, null pointer exceptions, etc. Addressing these issues early in development helps the teams to avoid problems later.

Improved code quality: Static testing helps to make sure that the code is easy to maintain and well-structured. This leads to a higher quality code.

Reduced costs: Early bug detection in static testing helps to fix them early in the development thus saving time, effort, and cost.

Immediate feedback: Static testing provides immediate evaluation and feedback on the software during each phase while developing the software product.

Helps to find exact bug location: Static testing helps to find the exact bug location as compared to dynamic testing.

Limitations/disadvantages of Static Testing

1. Detect Some Issues: Static testing may not uncover all issues that could arise during runtime. Some defects may appear only during dynamic testing when the software runs.

Depends on the Reviewer's Skills: The effectiveness of static testing depends on the reviewer's skills, experience, and knowledge.

2. Time-consuming: Static testing can be time-consuming when working on large and complex projects.

3. No Runtime Environment: It is conducted without executing the code. This means it cannot detect runtime errors such as memory leaks, performance issues, etc.

4. Prone to Human Error: Static testing is prone to human error due to manual reviews and inspections techniques being used.

Dynamic Testing/ Structural Techniques:

Dynamic testing is a type of software testing that involves executing the software and evaluating its behavior during runtime. It is also known as functional testing.

Dynamic testing performed to analyze the dynamic behavior of the code. It includes the testing of the software for the input values and output values that are analyzed. It is performed to describe the dynamic behavior of code. It refers to the observation of the physical response from the system to variables that are not constant and change with time.

We can do dynamic testing by following techniques

1. Statement Coverage: It ensures every executable statement in the code is executed at least once.

Based on the input given to the program, some statements are executed and some may not be executed So,

This technique is used to calculate total number of executable statements in the code.

For that following formula is used,

$$\text{Statement coverage} = \frac{\text{Number of executed statements}}{\text{Total Number of statements}} \times 100$$

Consider following source code,

```
1. a=10,b=20;
2. if(a>b)
3. {
4.     printf("a is greater");
5. }
6. else
7. {
8.     printf("a is greater");
9. }
```

Let's consider the input scenario, a=10,b=20 then the statement coverage is

Total number of statements=9

Total number of executable statements=5

So we can calculate statement coverage as,

$$\text{Statement coverage} = \frac{5}{9} \times 100$$

Statement coverage=55.55 %

2. Branch Coverage:

It is WBT technique used to cover all branches of CFG(Control Flow Graph).

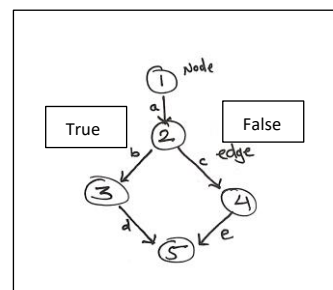
It Ensures every possible outcome of each decision point (e.g., if/else statements, switch cases) is executed.

To calculate the branch coverage, Number of executed paths are considered.

Consider the following example,

```
1 input a=10,b=5
2 if(a>b) Then
3 print a is large
4 else print b is large
5 end
```

CFG for this code=>



Here,

Nodes=1,2,3,4,5

Edges=a,b,c,d,e

Paths are- I) 1-->2-->3-->5

II) 1-->2-->4-->5

By above CFG when the condition is true Then the path is 1-->2-->3-->5. This path covers edges **a,b,d** .

when the condition is false Then the path is 1-->2-->4-->5. This path covers edges **a,c,e** .

By traversing both paths we can cover all edges i.e. branches So our branch coverages is 2

We can write as follows

Sr. No	Case	Covered Branches	Path	Branch Coverage
1	a=10,b=5 i.e. True	a,b,d	1-->2-->3-->5	2
2	a=5,b=10 i.e. False	a,c,e	1-->2-->4-->5	

3. Path Coverage: This technique also finds all the possible paths through the control flow of program.

In path testing each and every path in the program should be tested at least once.

There are three types of path testing

Steps for path testing

I) draw CFG

II) Determine cyclomatic complexity

III) find a basis set of path

IV) Generate Test case for each path

Consider the following example and apply steps of path testing

```
If A=50
Then if B>C
Then A=B
Else A=C
ENDIF
ENDIF
Print A
```

Step 1: Draw CFG as,

-Arrow or edges indicates flow of control

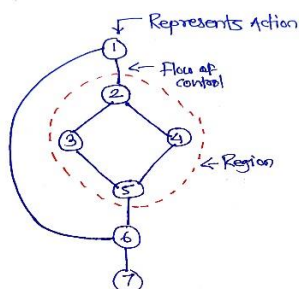
-Circles or nodes represent actions/statements of program

-areas bounded by edges and nodes which are further called as regions

-a predicate node is a node contains conditions.

```
1 If A=50
2 Then if B>C
3 Then A=B
4 Else A=C
5 ENDIF
6 ENDIF
7 Print A
```

CFG →



Step 2: Determine Cyclomatic complexity. (Cyclomatic complexity is a measurement developed by Thomas McCabe to determine the stability and level of confidence in a program.

It measures the number of linearly-independent paths through a program module. Programs with lower Cyclomatic complexity are easier to understand and less risky to modify)

We can measure cyclomatic complexity as follows,

$$\text{Cyclomatic complexity} = (\text{edges} - \text{nodes}) + 2P$$

Where

Edges: arrows / flows of control

Nodes: circle

P: Number of not connected parts of graph

By above GFG,

Edges=8, nodes=7 and p=1

Now let's calculate cyclomatic complexity as,

$$\begin{aligned}\text{Cyclomatic complexity} &= \text{edges} - \text{nodes} + 2P \\ &= (8 - 7) + 2 * 1 \\ &= 1 + 2 \\ &= 3\end{aligned}$$

Step 3: Find basis set of paths

The cyclomatic complexity tells us numbers of basis path present in graph as,

Path1 : 1-> 2-> 3-> 5-> 6-> 7

Path 2: 1-> 2-> 4-> 5-> 6-> 7

Path 3: 1-> 6-> 7

Step 4: Generate test cases for each basis path.

-While generating test cases we need at least one test case to cover one path.

-In above example path3 is covered by path1 and path 2

-So, here we have to generates 3 test cases(i.e. need to write test cases of path 1 and path 2)

Sr. No	Case	Covered Branches	Path
1	B>C i.e. True	a,b,d,f,g	1-->2-->3-->5-->6-->7
2	B>C i.e. False	a,c,e	1-->2-->4-->5-->6-->7

Advantages of Path Coverage Testing:

-Provides comprehensive code coverage, ensuring all possible execution paths are tested.

-Helps improve software quality and reliability by thoroughly testing all code paths.

-Utilizes a standardized metric, Cyclomatic Complexity, for assessing code complexity.

-Useful for demonstrating regulatory compliance in industries with strict requirements.

Disadvantages of Path Coverage Testing:

- Demands a high testing effort, particularly for complex code, leading to resource-intensive testing.
- Focuses on code paths but may not cover all potential runtime conditions or input combinations.
- Maintaining a comprehensive set of test cases as code evolves can be challenging.
- There is a risk of overemphasizing coverage quantity over quality, potentially neglecting lower-priority code paths.

4) Conditional Coverage:

Condition testing is another type of structural testing techniques.

- It is used for to test the logical conditions in the program.
- It is also called as predict testing in which each Boolean expression is predicated as True or false.
- It focuses on testing of each condition of the program.
- These conditions can be a Boolean condition or a relational expression.

The condition testing is used in program for following reasons

- 1) Boolean operator is incorrect or missing or extra
- 2) Boolean variable is incorrect.
- 3) Boolean parenthesis may be missing, incorrect or extra.
- 4) error in relational operator.
- 5) error in arithmetic expression.

Formula to calculate Conditional Coverage

$$\text{Condition Coverage} = (\text{Total count of conditions executed} / \text{Total count of conditions in the source code}) * 100$$

Example:

```
1 Input X, Y, Z, and W
2 IF (X == 0 || Y == 0)
3   THEN PRINT 100
4 ELSE IF (Z == 0 && W == 0)
5   THEN PRINT 200
6 END IF
7 END
```

Let us calculate the condition coverage with the first test case, with the inputs X = 0, Y = 0, Z = 0, W = 0.

With these values, the first condition of the OR operator with the expression X == 0, holds true. Since the result of the left side of the OR is already true, so the right side of the expression, Y == 0 of it, will be skipped from evaluation.

So, the condition Y == 0 remained unchecked. Then, the statement 3 will be executed resulting in printing 100 and the condition IF (Z == 0 && W == 0) in the ELSE part(at the line 4) will not be executed. Hence, out of a total four conditions, only one of them got executed.

The total condition coverage as per formula:

$$\text{Condition Coverage} = (1 / 4) * 100$$

$$\text{Condition Coverage} = 25 \%$$

Advantages of Conditional Coverage Testing

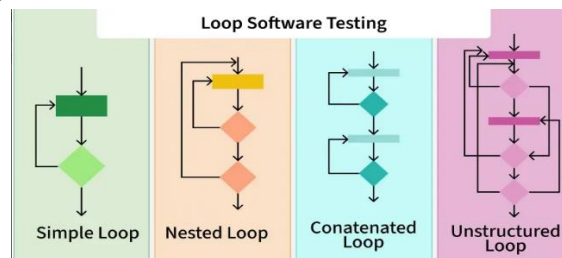
- The software condition coverage testing guarantees that all the conditions in the code are checked at least once.
- The software condition coverage testing detects defects from the early stages of the software development life cycle(SDLC).
- The condition coverage testing improves the quality, maintainability, and reliability of the software.
- The software condition coverage testing helps in faster troubleshooting of errors in the code.
- The condition coverage testing gives higher confidence, and trust on the code developed for the software.

Disdvantages of Conditional Coverage Testing

- If the boolean expression is large & complex then it is very tiresome job.

5) Loop Coverage: It Focuses on testing the behavior of loops, including zero iterations, one iteration, multiple iterations, and boundary conditions.

Types of Loop Coverage Testing



1. Simple Loop Testing

Testing performed in a simple loop is known as Simple loop testing. A simple loop is a normal (like for, while, or do-while) in which a condition is given and the loop runs and terminates according to the true and false occurrence of the condition, respectively. This type of testing is performed basically to test the condition of the loop whether the condition is sufficient to terminate loop after some point of time.

Example:

```
while(condition)
{
    statement(s);
}
```

2. Nested Loop Testing

Testing performed in a nested loop in known as Nested loop testing. Nested loop is basically one loop inside the another loop. In nested loop there can be finite number of loops inside a loop and there a nest is made. It may be either of any of three loops i.e., for, while or do-while.

Example:

```
while(condition 1)
{
    while(condition 2)
    {
        statement(s);
    }
}
```

3. Concatenated Loop Testing

Testing performed in a concatenated loop is known as Concatenated loop testing. It is performed on the concatenated loops. Concatenated loops are loops after the loop. It is a series of loops. Difference between nested and concatenated is that in nested loop is inside the loop but here loop is after the loop.

Example:

```
while(condition 1)
{
    statement(s);
}
while(condition 2)
{
    statement(s);
}
```

4. Unstructured Loop Testing

Testing performed in an unstructured loop is known as Unstructured loop testing. Unstructured loop is the combination of nested and concatenated loops. It is basically a group of loops that are in no order.

Example:

```
while()
{
    for()
    {}
    while()
    {}
}
```

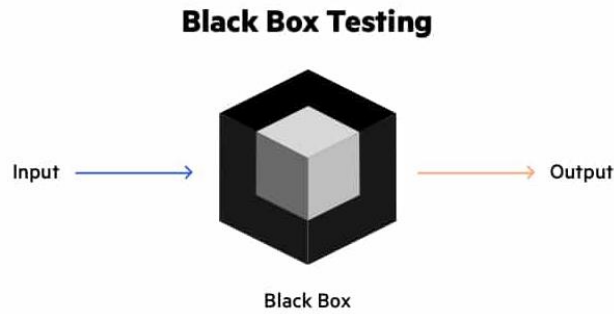
Advantages of Loop Testing

- Loop testing limits the number of iterations of loop.
- Loop testing ensures that program doesn't go into infinite loop process.
- Loop testing endures initialization of every used variable inside the loop.
- Loop testing helps in identification of different problems inside the loop.
- Loop testing helps in determination of capacity.

Disadvantages of Loop Testing

- Loop testing is mostly effective in bug detection in low-level software.
- Testing nested or unstructured loops requires extensive test case design, increasing effort.
- White-box technique, demands familiarity with the codebase.
- Loop testing is not useful in bug detection.

Introduction to Black Box Testing



Black box testing involves testing a system with no knowledge of its internal workings.

A tester provides an input, and observes the output generated by the system under test.

This makes it possible to identify how the system responds to expected and unexpected user actions, its response time, usability issues and reliability issues.

Black box testing is a powerful testing technique because it tests a system end-to-end.

Black Box Testing Advantages

- Testers do not require technical knowledge, programming or IT skills
- Testers do not need to learn implementation details of the system
- Tests can be executed by crowdsourced or outsourced testers
- Low chance of false positives
- Tests have lower complexity, since they simply model common user behavior

Black Box Testing Disadvantages

- Tests are Difficult to automate
- Requires prioritization, typically infeasible to test all user paths
- In BBT it is difficult to calculate test coverage
- If a test fails, it can be difficult to understand the root cause of the issue
- Tests may be conducted at low scale or on a non-production-like environment

Black Box Testing Techniques:

Following are the BBT techniques which can be applied on Functional/Non Function/Regression Testing

- 1) Boundary Value Analysis
- 2) Equivalence Class Partition,
- 3) State Transition Technique
- 4) Cause Effective Graph
- 5) Decision Table
- 6) Use Case Testing

1) Boundary Value Analysis :

Boundary Value Analysis (BVA) is a black-box test design technique where test cases are designed to test the boundary limits of input values, because defects often occur at the edges rather than in the middle of ranges.

The standard approach for BVA involves testing values:

At the minimum and maximum boundaries.

Just below the minimum boundary (invalid).
Just above the maximum boundary (invalid).
Just above the minimum boundary (valid).
Just below the maximum boundary (valid).

Example: Age Verification for a Service

Consider a web application that requires users to be between 18 and 60 years old, inclusive, to register.

Partition	Range	Status	Boundary Values to Test
Invalid (too young)	Ages < 18	Invalid	17
Valid	Ages 18–60	Valid	18, 19, 59, 60
Invalid (too old)	Ages > 60	Invalid	61

2) Equivalence Class Partition:

An equivalence partitioning(ECP) is a test design technique that is very popularly used in software testing. It enables the testers to divide input data into groups. It helps to minimize the total count of tests without compromising on achieving a good test coverage. It helps to handle large test data while testing the software.

Guidelines for Equivalence Partitioning :

- If the range condition is given as an input, then one valid and two invalid equivalence classes are defined.
- If a specific value is given as input, then one valid and two invalid equivalence classes are defined.
- If a member of set is given as an input, then one valid and one invalid equivalence class is defined.
- If Boolean no. is given as an input condition, then one valid and one invalid equivalence class is defined.

Example:

Let us consider an example of any college admission process. There is a college that gives admissions to students based upon their percentage.

Consider percentage field that will accept percentage only between 50 to 90 %, more and even less than not be accepted, and application will redirect user to an error page. If percentage entered by user is less than 50 %or more than 90 %, that equivalence partitioning method will show an invalid percentage. If percentage entered is between 50 to 90 %, then equivalence partitioning method will show valid percentage.

Percentage * Accepts Percentage value between 50 to 90

Equivalence Partitioning		
Invalid	Valid	Invalid
<=50	50-90	>=90

3) State Transition Technique:

The state transition testing comes under the black box testing and is performed to verify the characteristics of the software for multiple sets of input conditions that are given to it in a specific order. It includes verification of both positive and negative flows.

In other words,

The state transition testing is conducted to verify the change in states of a software under varying inputs.

Objectives of Software State Transition Testing

The objectives of the state transition testing are listed below –

- The state transition testing is done to detect multiple phases of the software. These phases are interlinked to specific criteria's within the software.
- The state transition testing assists in creating the state transition diagram that describes the software's multiple phases and their changes.
- The state transition testing helps to verify if the software moves to proper states under different circumstances.
- The state transition testing validates if the starting and ending states of the software are correct.
- The state transition testing is conducted to check how the software reacts to unexpected circumstances.

Parts of Software State Transition Testing

The parts of the software state transition testing are listed below –

States – They are denoted by rectangles with rounded corners and describe the conditions of a software. Each state represents a node in the state transition diagram, where one node points to a specific state/condition.

Transitions – They are denoted by arrows and are used to indicate changes from one state to another when the software reacts to an event.

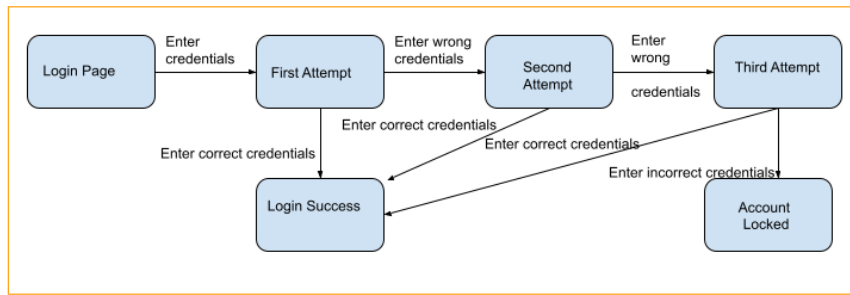
Events – They are labeled above the transition arrows. An event is an activity which brings in change in the software state.

Actions – They are denoted by message boxes. An action is a characteristic that the software generates when its state is modified.

Example

Let us take an example of a banking application where we would create a state transition diagram on login module features listed below –

- User enters correct credentials in the first attempt, user logs into the banking system.
- User enters correct credentials in the second attempt, user logs into the banking system.
- User enters correct credentials in the third attempt, user logs into the banking system.
- User enters correct credentials in the four attempts, user credentials are locked.



The total number of test cases that can be designed from the above state transition diagram are listed below –

Test Case 1 – User is in the Login Page, and then he enters the correct credentials in the first attempt, resulting in the successful login.

Test Case 2 – User is in the Login Page, and then he enters the correct credentials in the second attempt, resulting in the successful login.

Test Case 3 – User is in the Login Page, and then he enters the correct credentials in the third attempt, resulting in the successful login.

Test Case 4 – User is in the Login Page, and then he enters the incorrect credentials in the third attempt, resulting in the account locked.

4) Cause Effective Graph:

Cause Effect Graphing based technique is a technique in which a graph is used to represent the situations of combinations of input conditions. The graph is then converted to a decision table to obtain the test cases. Cause-effect graphing technique is used because boundary value analysis and equivalence class partitioning methods do not consider the combinations of input conditions.

Basic Notations used in Cause-effect graph: Here **c** represents cause and **e** represents effect.

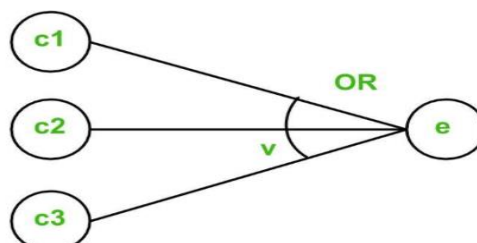
a) Identity Function: if c is 1, then e is 1. Else e is 0.



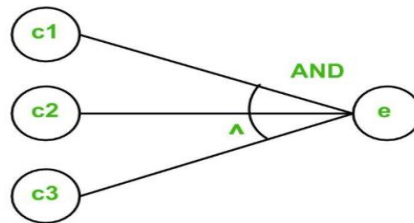
b) NOT Function: if c is 1, then e is 0. Else e is 1



c) OR Function: if c1 or c2 or c3 is 1, then e is 1. Else e is 0

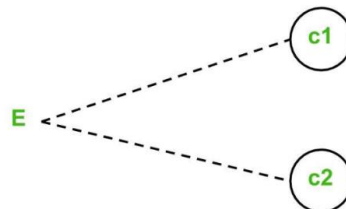


d) AND Function: if both c1 and c2 and c3 is 1, then e is 1. Else e is 0

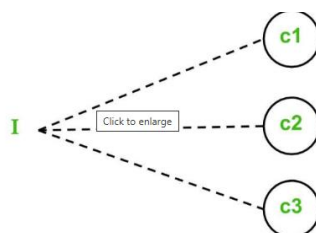


To represent some impossible combinations of causes or impossible combinations of effects, constraints are used. The following constraints are used in cause-effect graphs:

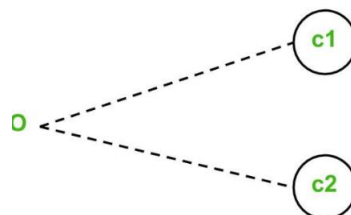
a) Exclusive constraint or E-constraint: This constraint exists between causes. It states that either c1 or c2 can be 1, i.e., c1 and c2 cannot be 1 simultaneously.



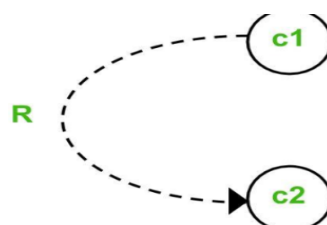
b) Inclusive constraint or I-constraint: It states that atleast one of c1, c2 and c3 must always be 1, i.e., c1, c2 and c3 cannot be 0 simultaneously.



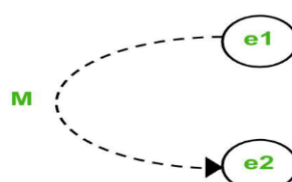
c) One and Only One constraint or O-constraint: These constraints are between the causes C1, and C2, such that one and only one of C1 and C2 should be 1.



d) Requires constraint or R-constraint: These constraints are between the causes C1, and C2, such that if C1 is equal to 1, then C2 should also be 1. It is not possible for C1 to have the value 1 with the C2 having the value as 0.



e) Mask constraint or M-constraint: These constraints are between the effects E1, and E2, such that if E1 is equal to 1, then E2 should be 0



Example

Let us take an example of a problem statement which says the first column should have the value X or Y. The second column should hold only digits. If both these conditions are satisfied, then a file should be modified. If the first column does not hold the values either X or Y, then the message M1 is generated. If the second column is fed with no digits, then the message M2 is generated.

To solve this problem using a control flow graph, we would first summarize the different conditions, and the events.

Condition 1 denoted by C1 refers to column 1 holding value X.

Condition 2 denoted by C2 refers to column 1 holding value Y.

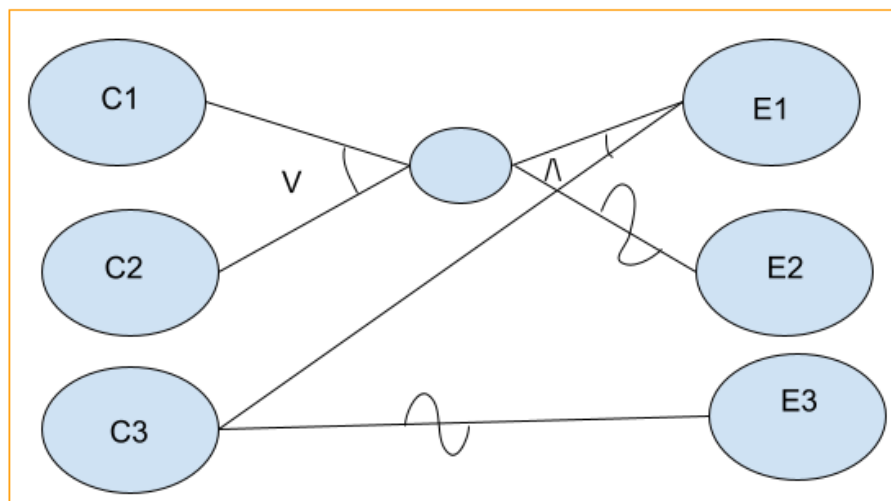
Condition 3 denoted by C3 refers to column 2 holding a digit.

Effect 1 denoted by E1 refers to modification of a file. The effect E1 should come true for the condition $\rightarrow (C1 \vee C2) \wedge C3$.

Effect 2 denoted by E2 refers to the message M1.

Effect 3 denoted by E3 refers to the message M2.

The cause effect graph of the above problem statement is given below –



5) Decision Table: A Decision table testing is a type of software testing that examines how a system responds to various input combinations. This is a methodical methodology in which the various input combinations and the accompanying system behavior (Output) are tabulated. That's why it's also known as a Cause-Effect table because it captures both causes and effects for improved test coverage.

The decision table allows testers to examine all conceivable combinations of requirements for testing and to immediately discover any circumstances that were overlooked. True(T) and False(F) values are used to signify the criteria.

Example: How to Create a Login Screen Decision Base Table

Let's make a login screen with a decision table. A login screen with E-mail and Password Input boxes.

The condition is simple – The user will be routed to the homepage if they give the right username and password. An error warning will appear if any of the inputs are incorrect.

Conditions	Rule 1	Rule 2	Rule 3	Rule 4
Username (T/F)	F	T	F	T
Password (T/F)	F	F	T	T
Output (E/H)	E	E	E	H

Legend

T - Make sure your login and password are correct.

F - Incorrect login or password

E - An error message appears.

H - The home screen appears.

Interpretation

Case 1 – Both the username and password were incorrect. An error message is displayed to the user.

Case 2 – The username and password were both right, however, the password was incorrect. An error message is displayed to the user.

Case 3 – Although the username was incorrect, the password was accurate. An error message is displayed to the user.

Case 4 – The user's username and password were both accurate, and the user went to the homepage.

We may generate two situations when converting this to a test case.

1) Enter the right username and password and click Login; the intended consequence is that the user will be sent to the homepage.

2) And one from the situation below.

-If the user types in the erroneous username and password and then clicks Login, the user should receive an error message.

-When you provide the proper username and incorrect password and click Login, the user should see an error message.

-If the user types the erroneous username and password and then clicks Login, the user should receive an error message.

6) Use Case Testing:

What is Use Case:

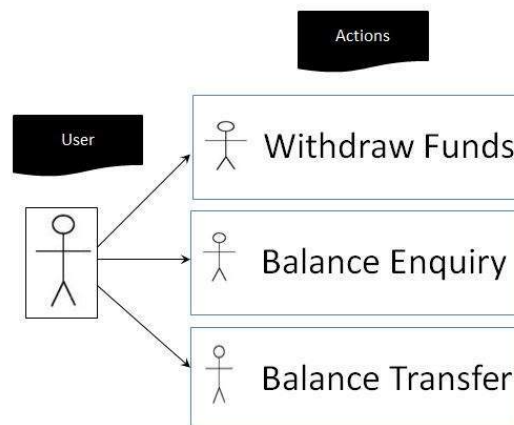
A Use Case is a tool for defining the required user interaction and if you are trying to create a new application or make changes to an existing application, several discussions are made.

- Use case depends upon the user's actions and the response of the system to the user's actions.
- Use cases are user-oriented, not system-oriented.
- Use case is a list of actions that will be taken by the actor while interacting with the system.
- It describes the functional requirements of the system.
- They help to uncover the integration defects, defects that arise due to incorrect interaction between different components.
- Use cases are used to hearten designers to outcomes before attempting to specify outcomes, and thereby
- they help to make requirements more proactive in system development.

What is Use Case Testing?

Use Case Testing is generally a part of black box testing and that helps developers and testers to identify test scenarios that exercise the whole system on each transaction basis from start to finish. Business experts and developers must have a mutual understanding of the requirement, as it's very difficult to achieve.

The Below example clearly shows the interaction between users and possible actions.



Types Of Black Box Testing

Black box testing are of three main types as: Functional, Non-functional and Regression testing.

Functional Testing

Black box testing can test specific functions or features of the software under test. For example, checking that it is possible to log in using correct user credentials, and not possible to log in using wrong credentials.

Functional testing can focus on the most critical aspects of the software (smoke testing/sanity testing), on integration between key components (integration testing), or on the system as a whole (system testing).

Types of Functional Testing

Functional Testing will be divided into further types which is following:

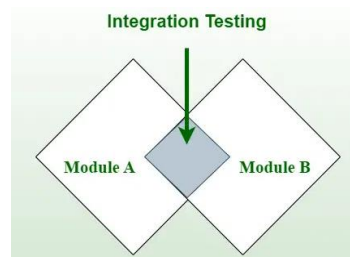
1. Unit Testing

Unit testing is the process of testing the smallest parts of the code, like it is a method in which we verify the code's correctness by running one by one.

It's a key part of software development that improves code quality by testing each unit in isolation.

Advantages	Disadvantages
* Early detection and resolution of issues.	* May not detect integration or system-level problems.
* Improved code quality.	* Requires significant time for writing and maintaining tests.
* Facilitates safer code refactoring.	* Tests require updates as code changes.
* Tests act as documentation.	* Requires skill and experience.
* Reduces overall development costs by catching errors early.	* Can lead to over-reliance, potentially neglecting other testing types.

2. Integration Testing



Integration Testing is a method of testing where different units or components of a software application interact with each other. It is used to identify and resolve any issues that may arise when different units of the software are combined.

The integration testing is conducted by selecting the unit by unit of a software in a logical sequence.

The goal of Integration Testing is to identify any problems or bugs that arise when different components are combined and interact with each other.

Pros: Early bug detection; easy to debug; thorough coverage.

Cons: Time-consuming due to the need for stubs and drivers.

We can divide integration testing in two types as

- a) Incremental Testing
- b) Non-Incremental Testing

a) Incremental Testing:

Incremental testing is a software testing strategy where modules are developed and integrated one by one, with immediate testing after each addition, rather than waiting for the whole system to be complete.

Further more the Incremental testing divided into following sub types as,

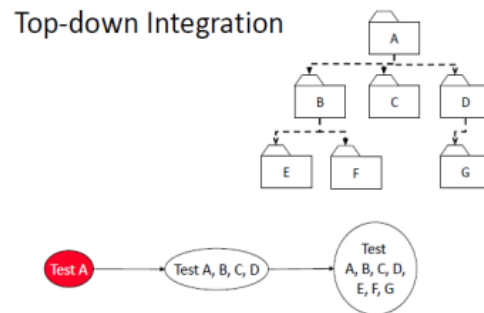
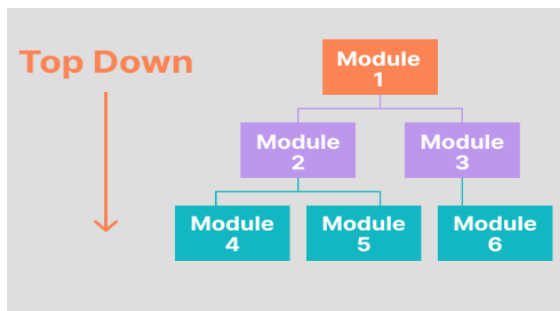
- i) Top Down
- ii) Bottom Up
- iii) Hybrid

i) Top Down :

In this type of testing, the high level units are tested first followed by the lower level units. After this process, the integration is considered as completed to ensure that the software is working as expected.

The drivers(drivers create interface between higher level modules) and stubs(Stubs create interface

between lower level modules) are developed to perform the software top down integration testing. It is used to enhance and stimulate the characteristics of the modules which are not yet combined into the lower levels.



- Testing will start from module 1 then proceed to module 2, module 3 etc.
- In top down approach only module 1 is tested in isolation rest all other modules are integrated one by one and tested.
- Consider Module 2 is not developed yet so in order to test module 1 we need module 2, so here use stubs.
- So in place of module 2 stubs are placed. This stub will accept input from module 1 and returns the result. Stubs can be created by tester.

Advantages of Top Down Integration Testing

- Separately debugged module.
- Few or no drivers needed.
- It is more stable and accurate at the aggregate level.
- Easier isolation of interface errors.
- In this, design defects can be found in the early stages.

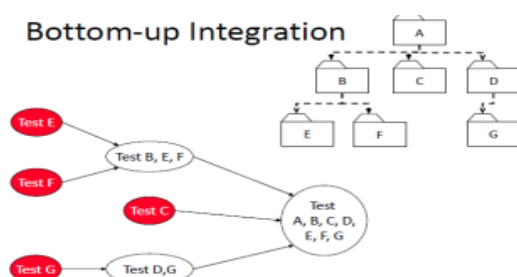
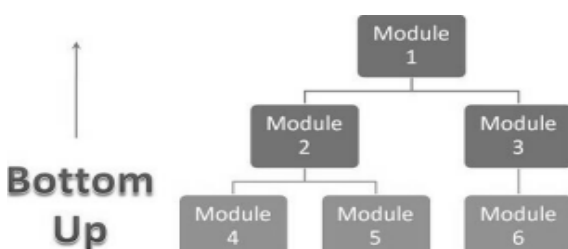
Disadvantages of Top Down Integration Testing

- Needs many Stubs.
- Modules at lower level are tested inadequately.
- It is difficult to observe the test output.
- It is difficult to stub design.

ii) Bottom Up

In bottom up Approach testing, testing takes place from bottom to top, i.e. modules at bottom layer are integrated and tested first and then sequentially other modules are integrated as upwards to the higher level modules. When higher level modules are missing or not developed which affects on the integration process. In this situation to make the integration process possible then developer develops the temporary dummy program is known as 'DRIVER'. Unavailable or not developed modules are replaced by drivers.

DRIVERS are the dummy programs which are used to call the functions of the lowest module in a case when the calling function does not exist.



Testing will start from module 4, module 5 and module 6 and this process will continue till modules are covered i.e. the process will come to halt once module 1 gets tested.

Advantages of Bottom-Up Integration Testing

- In bottom-up testing, no stubs are required.
- A principal advantage of this integration testing is that several disjoint subsystems can be tested simultaneously.
- It is easy to create the test conditions.
- Best for applications that uses bottom up design approach.
- It is Easy to observe the test results.

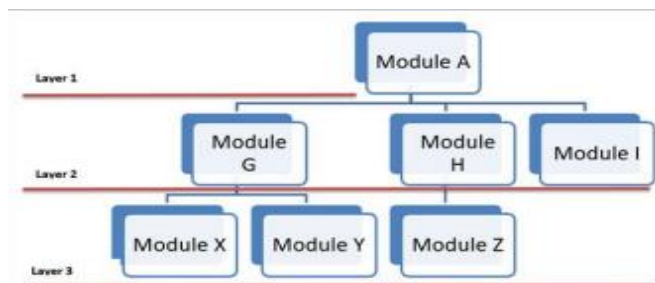
Disadvantages of Bottom-Up Integration Testing

- Driver modules must be produced.
- In this testing, the complexity that occurs when the system is made up of a large number of small – subsystems.
- As Far modules have been created, there is no working model can be represented.

iii) Hybrid (Sandwich) Testing

This approach is a hybrid of top-down and bottom-up methodology. Stub and drivers are used for incomplete or not developed modules.

- A middle layer is identified from which bottom-up and top-down testing are done. This middle layer is also known as target layer
- Target layer is identified as per Heuristic approach i.e., select a layer which allows minimal use of Stubs and drivers
- Top-down testing starts from middle layer and moves downwards towards lower level modules. This layer below middle layer is known as Bottom layer
- Bottom-up testing also starts from middle layer and move up towards top layer modules. This layer above middle layer is known as Top layer
- With use of stubs and drivers, user interface and functions of lower level modules are tested respectively. At the end, only middle layer is left for the execution of the final test



Testing Strategy:

- 1: Test A, X, Y, and Z individually – where Test A comes under Top layer test and Test X, Y and Z comes under Bottom layer tests
- 2: Test A, G, H and I
- 3: Test G, X, and Y
- 4: Test H and Z
- 5: Test A, G, H, I, X, Y, and Z

b) Non-Incremental Testing/ Big Bang Testing

Non-incremental testing, also known as big-bang testing, is a software testing approach where all components or modules of a system are integrated simultaneously, and the entire system is tested as a whole. Unlike incremental testing, which involves integrating and testing components or modules incrementally, non-incremental testing it takes entire system at once.

Advantages of non-incremental integration testing:

- This type of testing requires very little or no planning.
- All the modules are completed before the integration testing starts.

Disadvantages of non-incremental integration testing:

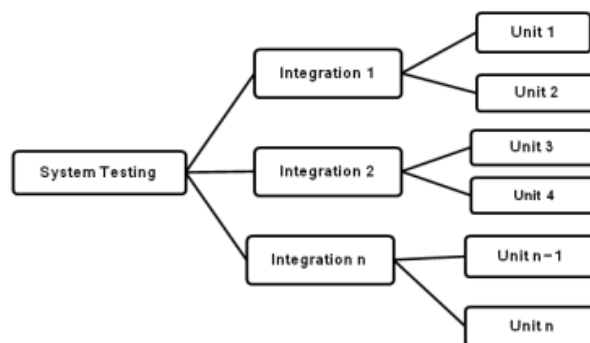
- Since all the modules are tested together chances of failure increases.
- Difficult to find the root cause of the failure if failure occurs.
- Difficult to detach the modules if any error occurs.
- Time consuming process.

3. System Testing

System testing, also known as end-to-end testing, it validates the complete and integrated software system against requirements.

It ensures all system components work as intended before the product goes live.

System testing verifies that the complete application works correctly in a real-world environment.



It is performed **after integration testing** and checks functional and non-functional aspects under real user conditions to ensure the system works as expected before release.

Advantages and Disadvantages of System Testing

Advantages	Disadvantages
Validates the complete system against business and technical requirements	Time-consuming due to the scope and depth of testing
Identifies issues in end-to-end workflows and data flows	Requires extensive test environments and setup
Ensures compatibility, performance, and security under real conditions	Difficult to trace the root cause of defects in large systems
Builds confidence in system readiness before release	Can delay delivery if major defects are found late

4) Acceptance Testing / User Acceptance Testing (UAT)

Acceptance testing is the final check in software development to ensure the product meets business goals and user expectations before release.

The user will test the system to find out whether the application meets their business needs.

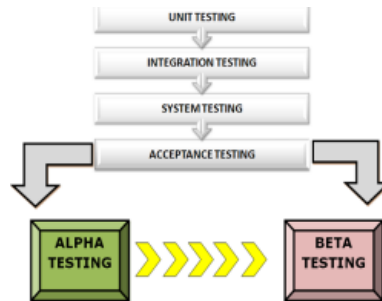
Once the acceptance test is passed then the application is released to production.

Acceptance testing done after System testing.

The Acceptance testing are mainly divided into two types as,

a) Alpha Testing

b) Beta Testing



a) Alpha Testing

Alpha testing is a type of acceptance testing; performed to identify all possible bugs before releasing the system.

Alpha testing is carried out in a lab environment and performed by the developers

Alpha testing is not open to market and public.

Advantages of alpha testing:-

- Provides view about software at an early stage.
- Provides early detection of errors.

Disadvantage of alpha testing:-

- Depth testing cannot be done as the project is under development.

b) Beta Testing (Field Testing)

Beta testing is testing of the product in the user environment. It is done at the client side and all end users use the system and see whether the system is working as per their given requirements.

Beta testing increases the quality of the product and reduces the failure rate of the product.

Beta testing is open for the market and public.

Beta version of the application will be released to limited number of end-users to in order to get feedback regarding the quality.

Advantages of Beta Testing:-

- Beta Testing improves the product quality via the customer feedback.
- With the received feedback, testers can fix all the issues and problems encountered by the end user before the product is released for public use.
- Users can install as well as test the application, and send their feedbacks to the developers, during beta testing.

Disadvantages of Beta Testing:-

- Testing is an issue because here the testing is done in the outside world of an organization.
- Finding the right beta tester is a difficult job.

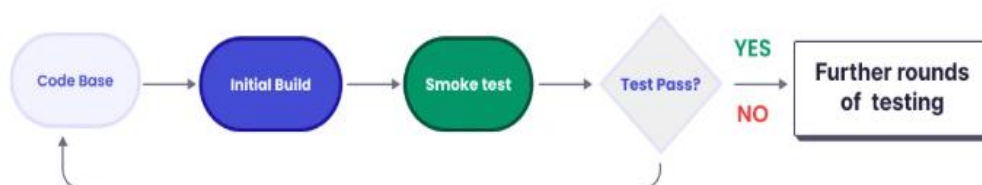
Alpha Testing	Beta Testing
Alpha testing uses both white box and black box testing.	Beta testing commonly uses black-box testing.
Alpha testing is performed by testers who are usually internal employees of the organization.	Beta testing is performed by clients who are not part of the organization.
Alpha testing is performed at the developer's site.	Beta testing is performed at the end-user of the product.
Reliability and security testing are not checked in alpha testing.	Reliability, security and robustness are checked during beta testing.
Alpha testing ensures the quality of the product before forwarding to beta testing.	Beta testing also concentrates on the quality of the product but collects users input on the product and ensures that the product is ready for real time users.
Alpha testing requires a testing environment or a lab.	Beta testing doesn't require a testing environment or lab.
Alpha testing may require a long execution cycle.	Beta testing requires only a few weeks of execution.
Developers can immediately address the critical issues or fixes in alpha testing.	Most of the issues or feedback collected from the beta testing will be implemented in future versions of the product.
Multiple test cycles are organized in alpha testing.	Only one or two test cycles are there in beta testing.

Smoke testing

Smoke testing also known as "Build Verification Testing" or "Build Acceptance Testing,"

This testing is typically performed at the beginning of the development process to ensure that the most critical functions of a software application are working correctly.

It is used to quickly identify and fix any major issues with the software before more detailed testing is performed.



The Smoke testing is executed by QA engineers/QA lead/testers after every build is received for checking the build is in testable condition. **This type of testing is applicable in the Integration Testing, System Testing and Acceptance Testing levels.**

Why it is called 'Smoke' testing?

Smoke testing got its name from testing of electrical appliances. While testing an electrical device was to plug it to a power outlet and see if it produces smoke or not.

If there is no smoke then it means that the electrical circuits are not faulty and device can be tested for its functionality. In "Software Smoke tests" we check that the bare minimum functionality is working or not.

Example of Smoke Testing - FACEBOOK with set of Smoke Test cases:

- User is able to Load the application
- Application is responsive enough
- User is able to login
- User can create, edit, delete his profile
- User is able to view other profiles, groups, pages etc.
- User is able to create groups, pages etc.
- Chat functionality is working
- User is able to view the notification
- User is able to upload the pictures
- User is able to share updates

Non-Functional Testing:

Non-functional testing focuses on evaluating the system's performance, scalability, security, usability, and reliability, rather than its specific functionality. It ensures that the system can handle real-world demands and provides a seamless, high-quality user experience.

Advantages of Non-functional Testing

- Improved performance: Non-functional testing checks the performance of the system and determines the performance bottlenecks that can affect the performance.
- Less time-consuming: Non-functional testing is overall less time-consuming than the other testing process.
- Improves user experience: Non-functional testing like Usability testing checks how easily usable and user friendly the software is for the users. Thus, focus on improving the overall user experience for the application.
- More secure product: As non-functional testing specifically includes security testing that checks the security bottlenecks of the application and how secure is the application against attacks from internal and external sources.

Disadvantages of Non-functional Testing

- Non-functional tests are performed repeatedly: Whenever there is a change in the application, non-functional testing is performed again. Thus, it is more resource intensive.
- Expensive in case of software update: In case of software update, non-functional testing is performed again thus incurring extra charges to re-examine the software, and thus software becomes expensive.

Types of Non-Functional Testing

- a) Performance Testing b) Usability Testing c) Compatibility Testing

a) Performance Testing

It is a testing technique carried out to determine system performance in terms of sensitivity, reactivity, and stability under a particular workload.

Performance testing is a type of software testing that focuses on evaluating the performance and scalability of a system or application.

Performance testing aims to measure system performance under various loads and conditions, and ensure that the system can handle the expected number of users or transactions.

Types of performance testing

- i) Load Testing ii) Stress Testing iii) Volume Testing iv) Soak Testing
- v) Recovery Testing vi) Scalability Testing

i) Load Testing:

Load testing determines the behavior of the application when multiple users use it at the same time. It is the response of the system measured under varying load conditions.

The load testing is carried out for normal and extreme load conditions.

- Load testing is a type of performance testing that simulates a real-world load on a system or application to see how it performs under stress.
- The goal of load testing is to identify bottlenecks and determine the maximum number of users or transactions the system can handle.
- During load testing, various scenarios are simulated to test the system's behavior under different load conditions

ii) Stress Testing

Stress Testing pushes the system beyond its normal load capacity to assess its breaking point.

This helps identify how the system behaves under extreme traffic and how gracefully it recovers from failure.

The primary focus of stress testing is to identify application bugs that occur in high load conditions.

And the Stress conditions be like

- Closing and restarting database access suddenly
- Randomly shutting down network connections
- Severely reducing physical resources such as disk space, memory or network bandwidth

iii) Volume Testing

Volume Testing Focuses on how the system performs when handling large volumes of data. It's used to identify performance issues related to database size, file uploads, or data processing at scale.

Volume testing will to verify the physical and logical limits to a system's capacity.

For example, testing the music site behavior when there is millions of user to download the song.

In volume testing, following things need to be tested

- Test to check if there is any data loss
- Check the system's response time
- Check if the data is stored correctly or not
- Verify if the data is overwritten without any notification
- Check for warning and error messages, whether it comes at all for volume problems
- Check whether high volume data affects the speed of processing

iv) Soak Testing /Endurance Testing

Soak Testing Examines system performance over an extended period under a steady load. It helps uncover memory leaks, performance degradation, and other long-term reliability issues.

For Example ,A system may behave normally when used for 2 hours, but when the same system is used continuously for 10 hours or more than that then it may fail or behave abnormally/randomly/it may crash. To predict such failure Soak Testing is performed.

v) Recovery Testing

Recovery Testing verifies software's ability to recover from failures like software/hardware crashes, network failures etc. The purpose of Recovery Testing is to determine whether software operations can be continued after failure or integrity loss. Recovery testing involves reverting back software to the point where integrity was known and reprocessing transactions to the failure point.

A system or software should be recovery tested for failures like:

- Power supply failure
- The external server is unreachable
- Wireless network signal loss
- Physical conditions
- The external device not responding
- The external device is not responding as expected, etc.

vi) Scalability Testing

Scalability Testing is a type of non-functional testing in which the performance of a software application, system, network or process is tested in terms of its capability to scale up or scale down the number of user request load or other such performance attributes. It can be carried out at a hardware, software or database level.

Regression Testing

Regression testing means we develop an application and test it to check whether its functions are working fine. Now, we make changes to it and again we need to test to check whether the functions are still working perfectly in addition to those new changes. i.e. retesting the unchanged parts of the application. “The main aim of regression testing to make sure that changed component is not impacting the unchanged part of the component”.

Regression Testing Techniques

There are three Regression Testing Techniques which are as below

Retest All

- In which all the tests in the existing test suite should be re-executed.
- This is very expensive as it requires huge time and resources.

Regression Test Selection

- Instead of re-executing the entire test suite, it is better to select part of test suite to be run
- Test cases selected can be categorized as 1) Reusable Test Cases 2) Obsolete Test Cases.
- Re-usable Test cases can be used in succeeding regression cycles.
- Obsolete Test Cases can't be used in succeeding cycles.

Prioritization of Test Cases

- Prioritize the test cases depending on business impact, critical & frequently used functionalities.
- Selection of test cases based on priority will greatly reduce the regression test suite.

Types of regression testing

a) Unit regression –

Testing the only one particular fixed unit or feature .Unit regression testing, executed during the unit testing phase, tests the code as a single unit. It has a narrow and focused approach, where complex interactions and dependencies outside the unit of code in question are temporarily blocked.

b) Regional or Partial regression –

Testing the changes and the impact regions is called Regional Regression testing. Partial regression is performed after impact analysis. In this testing process, the new addition of the code as a unit is made to interact with other parts of older existing code, in order to determine that even with code modification, the system functions in silos as desired.

c) Full or Complete regression –

Complete regression testing is often carried out when the code changes for modification or the software updates seep way back into the roots. It is also carried out in case there are multiple changes to the existing code. It gives a comprehensive view of the system as a whole and weeds out any unforeseen problems. A sort of a “final” regression testing is implemented to certify that the build (new lines of code) has not been altered for a period of time. This final version is then deployed to end-users

Experience-based testing:

Experience-based testing is a practical software testing approach that relies on a tester's skills, intuition, and accumulated experience to identify potential defects. Unlike formal, documented methods, it focuses on human adaptability and creativity to uncover issues that might be missed by rigid procedures or automated testing.

Types of Experience-based testing

- a) Error guessing b) Exploratory testing c) Ad-hoc testing

a) Error guessing

Error guessing is an informal testing technique where testers rely on their experience, intuition, and domain knowledge to identify potential defects in software applications that may not be captured by formal test cases or specifications. It involves guessing where errors or bugs might exist based on experience with similar systems or applications, common pitfalls, or understanding of user behavior and expectations.

Advantages of Error Guessing Technique

- It is effective when used with other testing approaches.
- It is helpful to solve some complex and problematic areas of application.
- It figures out errors which may not be identified through other formal testing techniques.
- It helps in reducing testing times.

Disadvantages of Error Guessing Technique

- Only capable and skilled tests can perform.
- Dependent on testers experience and skills.
- Fails in providing guarantee the quality standard of the application.
- Not an efficient way of error detection as compared to effort.

b) Exploratory testing



Exploratory testing means testing of software without any specific plans and schedules.

This is formal testing process where we don't have any test cases or test planning documents to test the application.

Testers continuously make decision on the next step of action.

In exploratory testing, test planning, analysis, design and test execution, are all done together and instantly.

It completely depends upon the testers thought process.

Tester opens the application and just starts browsing it to identify defects.

The testing quality is also dependent upon the tester's knowledge about the product to design test cases and the different test methods for finding defects.

We can perform the test design and activities parallel typically without formal document, the test cases, test conditions or test scripts.

Advantages

- It doesn't require preparation for testing as we don't have documents for testing.
- In this type of testing time saves due to all task are doing simultaneously like Testing, Designing test scenarios and executing test scenarios.
- Tester can report many issues due to incomplete requirement or missing requirement document.

Disadvantages

- Few issues cannot be catch in this type of testing.
- There is review of test planning & designing of test cases while testing may cause issues.
- Testers have to remember the scenario what he is executing because if any bug is found then tester should report a bug with proper steps to reproduce Difficulty to perform the exact manner especially for new found bugs.

c) Ad-hoc testing

Adhoc testing is a type of software testing that is performed informally and randomly

This testing also done without any specific plans and schedules.

Adhoc testing is not performed in a structured way so it is not based on any methodological approach.

That's why Adhoc testing is a type of Unstructured Software Testing.

Types of Ad-hoc Testing:

- i) Buddy Testing ii) Pair Testing iii) Monkey Testing

i) Buddy Testing

Buddy testing is a type of Adhoc testing where two persons will be involved one is from the Developer team and one from the tester team.

So that after completing one module and after completing Unit testing the tester can test by giving random inputs and the developer can fix the issues too early based on the currently designed test cases.

ii) Pair Testing

In Pair testing two persons from the testing team can be involved to test the same module.

When one tester can perform the random test another tester can maintain the record of findings.

So when two testers get paired they exchange their ideas, opinions, and knowledge so good testing is performed on the module.

iii) Monkey Testing

Monkey testing is a type of Adhoc testing in which the system is tested based on random inputs without any test cases the behavior of the system is tracked and all the functionalities of the system are working or not is monitored. As the randomness approach is followed there is no constraint on inputs so it is called Monkey testing.

Advantages of Adhoc testing

- The errors that can not be identified with written test cases can be identified by Adhoc testing.
- It can be performed within a very limited time.
- Helps to create unique test cases.
- This test helps to build a strong product that is less prone to future problems.

Disadvantages of Adhoc testing

- Sometimes resolving errors based on identified issues is difficult as no written test cases and documents are there.
- Needs good knowledge of the product and testing concepts to perfectly identify the issues in any model.
- It does not provide any assurance that the error will be identified.
- Finding one error may take some uncertain period.

Testing Type	Scope	Purpose
Unit Testing	Smallest unit (function/method)	Validates individual components in isolation
Integration Testing	Multiple units combined	Ensures modules interact correctly
System Testing	Entire application	Tests full software functionality
Acceptance Testing	End-user perspective	Confirms business requirements are met